

1 System Overview

Two-project Dagster code location system. The **dbt canonical pipeline** builds a canonical dimensional model on top of the source trading system (SQL Server), materialised as Iceberg tables in Dremio. The **Dagster hub** is a central K8s orchestration control plane that federates the canonical pipeline and other data projects via gRPC, providing a single UI, PostgreSQL-backed run history, and K8s job execution across all locations.

Data Source — SQL Server

- Source system schema
- 105+ registered source tables
- Trading: trades, deals, parcels, voyages
- Financial: invoices, claims, swaps, options
- Reference: currencies, countries, desks, grades
- Read via Dremio SQL Server connector

dbt Canonical Pipeline — Code Location

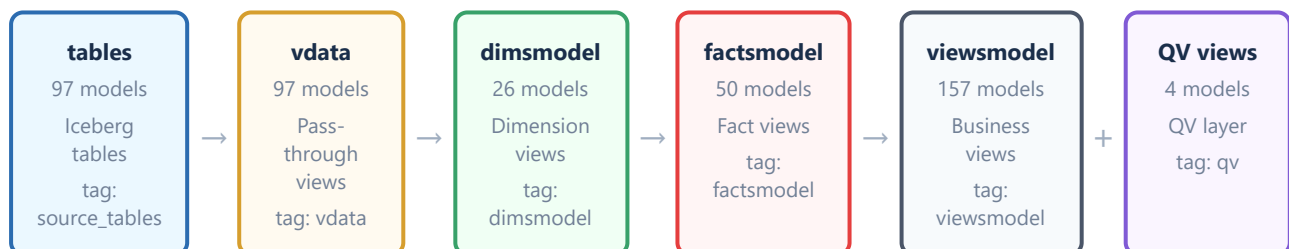
- dbt-core 1.10.20 + dbt-dremio 1.10.0
- Dagster 1.12.21 — gRPC code server, port 4004
- 431 dbt models across 5 layers
- 8+ Dagster jobs, 5 schedules, 1 failure sensor
- Canonical SQL views (27 dims, 51 facts)
- Test harness + email reporting

Dagster Hub — Orchestration Control Plane

- Dagster 1.12.21 — K8s webserver, port 3000
- Federates 3–6 code locations via gRPC
- PostgreSQL 15 — run/event/schedule storage
- K8sRunLauncher — spawns K8s jobs per run
- QueuedRunCoordinator — max 10 concurrent
- DagsterDaemonScheduler

2 dbt Pipeline Layers

Five sequential dbt layers materialised in Dremio. Source tables are loaded as Iceberg tables (S3-backed), bridged through vdata pass-through views, then transformed into dimensional and fact views for analytics consumption.



Two-Layer Design

canonical_dm/v4/ — Raw Dremio SQL views (source of truth for business logic). Direct table refs like `SourceDB.schema.tblTradesHeader`. Not dbt Jinja.

models/ — dbt execution layer wrapping canonical logic. Uses `{{ ref() }}` and `{{ source() }}` for Dagster asset graph construction and lineage.

Execution Order & Configuration

Sequential: tables → vdata → dimsmodel → factsmodel → viewsmodel

dbt config: 8 threads, profiles.yml for dev/qa/preprod/prod targets. Dremio port 9047 (SSL). Object storage: S3. Schema: materialize (tables), canonical.v4.dims / canonical.v4.facts (views).

3 Dagster Jobs & Schedules

Job	Layer / Tag
tables_only	source_tables
vdata_only	vdata
v4_dims_only	dimsmodel
v4_facts_only	factsmodel
v4_views_only	viewsmodel
qv_views_only	qv + views
sequential_12h	Full pipeline (all layers)
test_harness_email_report_job	Test harness + email

Schedule	Cron	Status
sequential_12h_schedule	0 */12 * * *	RUNNING
qv_views_hourly_schedule	0 * * * * *	RUNNING
test_harness_email_report_schedule	0 10,18 * * *	RUNNING
stage_audit_loader_schedule	30 0 * * *	STOPPED
test_scheduler	* * * * * *	STOPPED

Failure Sensor

`email_failure_sensor` (@dg.run_failure_sensor) — fires on any job failure across the code location. Sends HTML email via SMTP (port 25) to 9 configured recipients. SSL verification disabled (internal mail relay).

Materialization Report

After each successful run of report-enabled jobs, `success_report.py` emails a materialization summary (row counts, execution times per model). Triggered via `REPORT_ENABLED_JOBS` list in `definitions.py`.

4 Dagster Hub — Code Location Federation

The Dagster hub is a pure orchestration control plane with no assets of its own. It loads external code locations via gRPC, providing one unified UI, run history, and scheduler across all data projects in the organisation.

trading_canonical

:4000

Trading canonical model

orchestrator

:4001

Workflow orchestration

ref_data_elt

:4002

Reference Data ELT (dev only)

dbt-canonical-pipeline ★

:4004

This project — all envs

risk_pipeline

:4006

Risk calculations (dev only)

data_warehouse

:4007

Data warehouse (dev only)

Hub Instance Config

PostgreSQL 15 — event log, run storage, schedule storage.

K8sRunLauncher — spawns one K8s Job per pipeline run.

QueuedRunCoordinator — max 10 concurrent runs.

DagsterDaemonScheduler — polls every 30s.

Compute logs: `/apps/dagster/logs`.

K8s Executor Sizes

small — CPU 500m/1000m · Mem 1Gi/2Gi

medium — CPU 1000m/2000m · Mem 2Gi/4Gi

large — CPU 2000m/4000m · Mem 4Gi/8Gi

xlarge — CPU 4000m/8000m · Mem 8Gi/16Gi

Key Architecture Patterns



gRPC Code Location Federation

The Dagster hub loads each project as an external gRPC server (`workspace-{env}.yaml`). The canonical pipeline code location runs `dagster code-server start -m dagster_definitions.definitions -h 0.0.0.0 -p 4004` inside its own K8s pod. The hub connects at runtime — no code bundling required.



Two-Layer dbt Design

`canonical_dm/v4/` holds the source-of-truth Dremio SQL (raw, no Jinja). `models/` is the dbt execution wrapper using `{{ ref() }}` / `{{ source() }}`. Business logic changes happen in canonical SQL; dbt layer drives Dagster asset graph and lineage only.



Tag-Driven Job Selection

Every dbt model is tagged (`source_tables`, `vdata`, `dimsmodel`, `factsmodel`, `viewsmodel`, `qv`). Dagster jobs use `AssetSelection.tag()` — no manual asset enumeration. Adding a new model with the right tag automatically includes it in the correct job.



Iceberg-Backed Source Tables

97 source table models are materialised as Apache Iceberg tables on S3. This gives ACID transactions, time-travel, schema evolution, and efficient incremental loads. Downstream `vdata`, `dim`, and `fact` models read from Iceberg via Dremio views.



Audit Macro System

9 dbt macros provide audit logging: `stage_insert_audit_log`, `stage_audit_loader_to_audit_main`, `get_last_load_time`, `get_source_rowcount` / `get_target_rowcount`. Source vs. target row counts compared and logged per run for data quality tracking.



External Test Harness

`scripts/test_harness.py` validates Iceberg materialisations against SQL Server source: row count comparison (threshold %), column metadata check, SELECT accessibility. Outputs HTML/JSON report, optionally emailed. Runs as a scheduled Dagster job (10:00 & 18:00 UTC).

6 Technology Stack

Component	Technology / Version
dbt Core	dbt-core 1.10.20
dbt Adapter	dbt-dremio 1.10.0
Orchestration	Dagster 1.12.21 · dagster-dbt 0.28.21
Dagster K8s	dagster-k8s 0.28.21
Dagster PostgreSQL	dagster-postgres 0.28.21
Data Lakehouse	Dremio (dremio+flight, port 9047, SSL)
Table Format	Apache Iceberg (S3-backed)
Source System	SQL Server (source trading system schema)
Metadata DB	PostgreSQL 15 (Dagster hub)
Data Validation	great-expectations 1.15.1
Data Transport	pyarrow 23.0.1 · boto3 1.28.21
Python	3.12+
Container	Docker · Helm · Rancher K8s
CI/CD	GitLab (Sengrep → Podman build → Nexus → Rancher Helm deploy)

Setting	Value
Code server port	4004 (dbt pipeline gRPC)
Hub webserver port	3000
dbt threads	8
Total dbt models	431 (97+97+26+50+157+4)
Source tables	105+ (SQL Server)
Canonical dims	27 (canonical_dm/v4/dims/)
Canonical facts	51 (canonical_dm/v4/facts/)
Max concurrent runs	10 (QueuedRunCoordinator)
Schedules active	3 of 5
Pipeline frequency	Every 12h (full) / hourly (QV)
K8s executor	small / medium / large / xlarge
Environments	dev, qa, preprod, prod, dr